Le mode sommeil du PIC16F84A

Polo Antoine

1. INTRODUCTION

En temps normal, le PIC16F84A consomme quelques mA, mais il est possible de le faire basculer dans un mode de sommeil où il ne consommera plus que quelques µA. Dans ce mode, l'oscillateur s'arrête, le CPU et TMR0 sont donc figés. Les broches gardent cependant leurs états.

Le PIC se met en sommeil avec l'instruction `SLEEP()`. Les sources de réveil peuvent être :

- un reset sur MCLR ;
- un débordement du WDT ;
- une interruption sur RB0/INT ;
- un changement d'état sur RB4 à RB7 ;
- la fin d'une écriture dans l'EEPROM.

Lors du réveil, si `GIE` est égal à 0, il reprend juste après le `SLEEP`. S'il est égal à 1, il exécute l'instruction suivante puis saute dans l'ISR. Cette instruction est souvent un `NOP()`.

2. REGISTRES IMPORTANTS

Les registres utiles ici sont `OPTION_REG` et `INTCON`. Le registre `STATUS` peut également être utile pour savoir si le PIC a été mis en sommeil.

2.1. Registre de gestion des interruptions (INTCON)

Table 1. Bits du registre INTCON.

b7	b6	b5	b4	b3	b2	b1	b0
GIE	EEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF

- `GIE` : interrupteur général. À 0, aucune interruption n'est traitée, même si son propre bit d'activation est à 1. À 1, les sources activées peuvent interrompre le programme. Le PIC le met automatiquement à 0 en entrant dans la routine d'interruption, pour éviter qu'une interruption n'en interrompe une autre, et le remet à 1 en en sortant.
- `EEIE` : à 1, une interruption est déclenchée lorsqu'une écriture dans l'EEPROM se termine. Cette écriture est lente (plusieurs millisecondes), ce qui permet de continuer le programme pendant ce temps.
- `T0IE` : à 1, une interruption est déclenchée lorsque le compteur `TMR0` déborde, c'est-à-dire lorsqu'il passe de 255 à 0.
- `INTE` : à 1, une interruption est déclenchée lorsqu'un front est détecté sur la broche RB0/INT. Le type de front (montant ou descendant) se choisit avec le bit `INTEDG` de `OPTION_REG`.
- `RBIE` : à 1, une interruption est déclenchée lorsque l'état d'une des broches RB4 à RB7 change, à condition qu'elle soit configurée en entrée. Le PIC compare l'état des broches à leur dernière lecture, sans distinguer front montant et descendant.
- `T0IF` : le matériel le met à 1 à chaque débordement de `TMR0`, que `T0IE` soit activé ou non. Il reste à 1 tant que le programme ne l'a pas remis à 0, ce qui doit être fait dans la routine d'interruption.
- `INTF` : le matériel le met à 1 lorsqu'un front du bon type est détecté sur RB0/INT. Il doit lui aussi être remis à 0 par le programme.
- `RBIF` : le matériel le met à 1 lorsqu'un changement d'état est détecté sur RB4 à RB7. Pour le remettre à 0, il faut d'abord lire (ou écrire) le PORTB, puis effacer le bit, sans quoi il se remet à 1 aussitôt.

Pour le mode sommeil, ce sont surtout `RBIE` et `RBIF` qui nous intéressent : un changement d'état sur RB4 à RB7 réveille le PIC, que `GIE` soit activé ou non.

2.2. Registre de configuration (OPTION_REG)

Table 2. Bits du registre OPTION_REG.

b7	b6	b5	b4	b3	b2	b1	b0
nRBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0

- `nRBPU` : contrôle les résistances de pull-up du PORTB ; le pull-up est activé à 0.
- `INTEDG` : sens de l'interruption externe (RB0), 1 pour front montant et 0 pour descendant.
- `T0CS` : source d'horloge du timer, 0 pour l'horloge interne et 1 pour l'horloge externe.
- `T0SE` : front d'horloge externe (si T0CS à 1), 0 pour front montant et 1 pour descendant.
- `PSA` : assignation du prédiviseur, à 0 il est assigné à Timer0 et à 1 pour le watchdog.
- `PS2-0` : détermine la valeur du prédiviseur. Il utilise une valeur binaire de 000 à 111.

Dans cet exemple, seul `nRBPU` nous intéresse : il active les pull-up du PORTB, ce qui permet de relier un bouton directement à la masse sur RB4 à RB7.

2.3. Registre d'état (STATUS)

Table 3. Bits du registre STATUS.

b7	b6	b5	b4	b3	b2	b1	b0
IRP	RP1	RP0	nTO	nPD	Z	DC	C

- `nTO` : indique si le démarrage est normal (1) ou un reset par le watchdog (0).
- `nPD` : passe à 0 si le PIC a été mis en veille via un `SLEEP()`, passe à 1 au démarrage.

La combinaison de ces deux bits permet de savoir pourquoi le PIC vient de démarrer, selon la Table 4.

Table 4. Cause du démarrage selon nTO et nPD.

nTO	nPD	Cause
1	1	Mise sous tension
0	1	Reset WDT en fonctionnement normal
0	0	Réveil par le WDT pendant SLEEP
1	0	Réveil par interruption ou MCLR pendant SLEEP

3. EXEMPLE DE CODE

3.1. Montage

Le montage reprend le circuit de base de l'introduction au PIC16F84A (quartz de 8 MHz, condensateurs de 22 pF et résistance de 10 kΩ sur MCLR), auquel on ajoute :

- une LED, avec sa résistance de limitation, sur la broche RB0 ;
- un bouton poussoir entre une des broches RB4 à RB7 (par exemple RB4) et la masse, sans résistance externe puisque le pull-up interne du PORTB est activé.

3.2. Code

Le code suivant permet de faire clignoter une LED, puis de passer le PIC en mode de sommeil. Il sera ensuite réveillé par une interruption sur RB4 à RB7, d'où la variable `dummy` qui est indispensable.

```
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config CP = OFF

#include <xc.h>

#define _XTAL_FREQ 8000000

#define LED_PIN PORTBbits.RB0

volatile uint8_t dummy;

void blink(uint8_t n);

void main(void) {
    PORTA = 0x00;
    TRISA = 0x00;
    PORTB = 0x00;
    TRISB = 0b11110000;

    OPTION_REGbits.nRBPU = 0;

    INTCONbits.RBIE = 1;

    while (1) {
        blink(5);

        LED_PIN = 0;
        dummy = PORTB;
        INTCONbits.RBIF = 0;

        SLEEP();
        NOP();
    }
}

void blink(uint8_t n) {
    for (uint8_t i = 0; i < n; i++) {
        LED_PIN = 1;
        __delay_ms(500);
        LED_PIN = 0;
        __delay_ms(500);
    }
}
```

Code 1. Mise en sommeil du PIC avec réveil sur RB4 à RB7.

3.3. Fonctionnement

- **Configuration** : `TRISB = 0b11110000` configure RB4 à RB7 en entrées, nécessaires pour l'interruption sur changement d'état, et RB0 à RB3 en sorties. Le PORTA est entièrement configuré en sorties. Définir toutes les broches non utilisées en sorties est une bonne pratique, car une entrée laissée flottante peut consommer du courant.
- **Activation** : `nRBPU = 0` active les pull-up du PORTB et `RBIE = 1` autorise l'interruption sur RB4 à RB7. `GIE` n'est pas activé : le PIC se réveille donc quand même, mais reprend simplement après le `SLEEP()` sans passer par une ISR.
- **Clignotement** : `blink(5)` fait clignoter la LED 5 fois, avec 500 ms allumée et 500 ms éteinte.
- **Préparation du sommeil** : la LED est éteinte avant de dormir, car les broches gardent leur état pendant le sommeil et une LED allumée consomme énormément par rapport au PIC. La lecture `dummy = PORTB` suivie de `RBIF = 0` efface le drapeau de changement d'état. La variable est `volatile` pour que le compilateur ne supprime pas cette lecture, qui ne sert à rien d'autre.
- **Sommeil et réveil** : `SLEEP()` met le PIC en sommeil. Un appui sur le bouton change l'état de la broche, ce qui le réveille. Il exécute alors `NOP()` puis la boucle recommence avec un nouveau clignotement.

4. CONCLUSION

Le mode sommeil réduit la consommation du PIC à quelques µA, ce qui est utile pour un montage sur pile. Il suffit d'autoriser une source de réveil, de remettre son drapeau à 0, de mettre les sorties dans un état qui ne consomme pas, puis d'appeler `SLEEP()`. Le registre `STATUS` permet ensuite de savoir pourquoi le PIC a redémarré.